Obsidian + Claude Agent — Segundo Cerebro con IA
Documentación Completa “For Dummies”
PARTE 1 — ¿De qué se trata todo esto?
El problema que resuelve
La idea central en una frase
¿Qué es un “segundo cerebro”?
PARTE 2 — Las herramientas que usa el proyecto
2.1 Obsidian — Tu cuaderno inteligente
2.2 VS Code — El editor de código
2.3 Claude Code — El cerebro artificial
2.4 Markdown — El lenguaje de las notas
PARTE 3 — Arquitectura del sistema (cómo encaja todo)
El flujo completo, paso a paso
El concepto clave: Context Engineering
PARTE 4 — Estructura de archivos del proyecto
El vault completo
PARTE 5 — Los archivos clave explicados uno por uno
5.1 CLAUDE.md — El system prompt permanente
5.2 .claude/settings.json — Los permisos del agente
5.3 La carpeta memory/ — La memoria del agente
PARTE 6 — Las plantillas explicadas
6.1 Plantilla de Nota Diaria (daily-note.md)
6.2 Plantilla de Proyecto (proyecto.md)
6.3 Plantilla de Persona (persona.md)
6.4 Plantilla de Research (research.md)
PARTE 7 — Cómo instalar y configurar el sistema
Requisitos previos
Instalación paso a paso
PARTE 8 — Cómo usarlo en el día a día
Flujo de trabajo típico
PARTE 9 — Conceptos técnicos para entender mejor
¿Qué es un LLM?
¿Qué es un agente de IA?
¿Qué es el context window?
¿Qué es YAML frontmatter?
¿Qué son los wikilinks?
PARTE 10 — Beneficios y casos de uso reales
¿Para qué tipo de persona es esto útil?
Casos de uso concretos
PARTE 11 — Buenas prácticas y recomendaciones
Para sacarle el máximo provecho
Lo que no hacer
PARTE 12 — Glosario rápido
PARTE 13 — Preguntas frecuentes
CONCLUSIÓN

Obsidian + Claude Agent — Segundo Cerebro con IA

Documentación Completa “For Dummies”

Autor del proyecto: Kevin Larguía
Repositorio: github.com/KevinLarguia/obsidian-claude-agent-IA-SecondBrain-
Licencia: MIT
Fecha de esta documentación: Abril 2026

PARTE 1 — ¿De qué se trata todo esto?

El problema que resuelve

Imaginá que tenés un cuaderno donde escribís ideas, proyectos, cosas que aprendés. Muy bien. Pero cada vez que abrís ese cuaderno con un asistente de IA (como ChatGPT o Claude), tenés que explicarle quién sos, en qué trabajás, qué proyectos tenés activos. Desde cero. Siempre.

Eso es frustrante y lento.

Este proyecto resuelve exactamente ese problema: construye un sistema donde la IA **ya sabe quién sos** cada vez que la usás. No tenés que re-explicar nada. El agente recuerda tu perfil, tus proyectos, tus preferencias de trabajo, y actúa en consecuencia desde el primer mensaje.

La idea central en una frase

> *"Un segundo cerebro digital donde la IA actúa como asistente personal que conoce tu contexto completo, sin que tengas que repetirte nunca."*

¿Qué es un "segundo cerebro"?

El concepto de "segundo cerebro" viene de la metodología de productividad personal: en lugar de tratar de recordar todo en tu cabeza, externalizás el conocimiento en un sistema digital. Anotás ideas, proyectos, aprendizajes, conexiones entre conceptos. Tu cabeza queda libre para pensar; el sistema guarda todo lo demás.

Este proyecto lleva eso un paso más allá: **el sistema no solo guarda, también entiende y actúa**.

PARTE 2 — Las herramientas que usa el proyecto

Antes de entender cómo funciona el sistema, necesitás conocer las piezas que lo componen.

2.1 Obsidian — Tu cuaderno inteligente

¿Qué es?
Obsidian es una aplicación gratuita para tomar notas. Sus notas son archivos de texto simples con formato Markdown (`.md`). La gracia es que podés crear **enlaces entre notas** usando doble corchete: `[[nombre-de-nota]]`.

¿Por qué es especial?

- Tus notas son archivos locales (no en la nube de otro, son tuyas)
- Podés crear una red de conocimiento donde las notas se conectan entre sí
- Es muy rápido y liviano
- Tiene un ecosistema enorme de plugins

Analogía: Obsidian es como una Wikipedia personal. Cada página puede linkear a otras páginas, y todo vive en tu computadora.

2.2 VS Code — El editor de código

¿Qué es?
Visual Studio Code (VS Code) es un editor de código gratuito de Microsoft. Aunque está pensado para programadores, en este proyecto se usa para que Claude Code tenga acceso a los archivos de tu vault.

¿Por qué se necesita?
Claude Code (la IA) funciona como una extensión dentro de VS Code. Desde ahí puede leer, escribir y editar tus archivos de Obsidian directamente.

2.3 Claude Code — El cerebro artificial

¿Qué es?
Claude Code es una extensión de VS Code (y también una herramienta de línea de comandos) creada por Anthropic. Es una versión de Claude —el modelo de IA de Anthropic— que tiene acceso directo a tus archivos locales.

¿Qué puede hacer?

- Leer archivos de tu vault
- Escribir y crear nuevas notas
- Editar notas existentes
- Buscar dentro de tus notas
- Entender el contexto de toda tu base de conocimiento

Analogía: Si Obsidian es tu biblioteca, Claude Code es el bibliotecario inteligente que sabe dónde está todo y puede agregar libros nuevos cuando se lo pedís.

2.4 Markdown — El lenguaje de las notas

¿Qué es?
Markdown es una forma simple de dar formato a texto usando símbolos especiales. Es lo que se usa en todas las notas del vault.

Ejemplos básicos:

```
# Título grande
## Título mediano
### Título pequeño

**texto en negrita**
*texto en itálica*

- elemento de lista
- otro elemento

[[enlace a otra nota]]

#etiqueta
```

No necesitás saber programar para usarlo. En 5 minutos lo entendés.

PARTE 3 — Arquitectura del sistema (cómo encaja todo)

El flujo completo, paso a paso

```
┌─────────────────────────────────────────────────────────┐
│                    TÚ (el usuario)                      │
│         Escribís algo en lenguaje natural               │
│    "Resumí lo que hice hoy y guardalo en mi nota"       │
└─────────────────────────────────────────────────────────┘
                        │
                        ▼
┌─────────────────────────────────────────────────────────┐
│                   CLAUDE CODE                           │
│    Lee automáticamente CLAUDE.md al iniciar             │
│    Entiende quién sos, tus proyectos, tus reglas        │
└─────────────────────────────────────────────────────────┘
                        │
                        ▼
┌─────────────────────────────────────────────────────────┐
│              TU VAULT DE OBSIDIAN                       │
│    Claude lee y escribe archivos .md                    │
│    Crea notas, actualiza proyectos, guarda ideas        │
└─────────────────────────────────────────────────────────┘
                        │
                        ▼
┌─────────────────────────────────────────────────────────┐
│                 MEMORIA PERSISTENTE                     │
│    Se guarda en carpeta memory/                         │
│    La próxima sesión, Claude vuelve a saber todo        │
└─────────────────────────────────────────────────────────┘
```

El concepto clave: Context Engineering

Context Engineering es la práctica de estructurar la información que le das a una IA para que opere de forma eficiente. En este proyecto, el archivo `CLAUDE.md` es el corazón del sistema.

Cuando Claude Code arranca, automáticamente lee `CLAUDE.md`. Este archivo es básicamente un **sistema prompt permanente** que le dice a Claude:

- Quién sos vos
- Qué proyectos tenés activos
- Cómo querés que se comporte
- Qué convenciones seguir
- Dónde guardar cada tipo de cosa

Es como darle a un nuevo empleado un manual de operaciones completo antes de que empiece a trabajar. Pero en este caso, lo hace cada vez que abrís una sesión.

PARTE 4 — Estructura de archivos del proyecto

El vault completo

```
vault/
│
├── CLAUDE.md                   ← El archivo más importante
├── .claude/
│   └── settings.json           ← Permisos del agente
│
├── memory/                     ← Memoria persistente de la IA
│   ├── MEMORY.md               ← Índice de memorias
│   ├── user_perfil.md          ← Tu perfil personal
│   ├── project_X.md            ← Info sobre proyectos
│   └── feedback_preferencias.md ← Cómo te gusta trabajar
│
├── daily-notes/                ← Una nota por cada día
│   ├── 2026-04-28.md
│   └── 2026-04-27.md
│
├── proyectos/                  ← Notas de tus proyectos
│   └── proyecto-ejemplo.md
│
├── personas/                   ← Perfiles de personas
│   └── nombre-apellido.md
│
├── ideas/                      ← Captura de ideas
│   └── idea-nueva.md
│
├── inbox/                      ← Entrada rápida sin organizar
│   └── captura-rapida.md
│
├── research/                   ← Investigación y aprendizaje
│   └── tema-de-estudio.md
│
├── resources/                  ← Recursos externos
│   └── libro-articulo.md
│
└── templates/                  ← Plantillas base
    ├── daily-note.md
    ├── proyecto.md
    ├── persona.md
    └── research.md
```

PARTE 5 — Los archivos clave explicados uno por uno

5.1 CLAUDE.md — El system prompt permanente

Este es **el archivo más importante del sistema**. Sin él, Claude es un asistente genérico. Con él, Claude se convierte en tu asistente personal.

¿Qué contiene?

Sección 1: Identidad del agente

Le dice a Claude cuál es su rol: > *"Actuás como asistente personal de segundo cerebro. Tu función es capturar, conectar y hacer crecer el conocimiento."*

Sección 2: Perfil del usuario
Información sobre vos: nombre, carrera, proyectos activos, objetivos. Esta sección la completás vos con tu información real.

Sección 3: Estilo de trabajo
Cómo trabajás: si preferís bloques de foco, si sos visual, si necesitás listas de tareas, etc.

Sección 4: Convenciones técnicas obligatorias
Reglas que Claude debe seguir siempre:

Convención	Ejemplo
Enlace interno	`[[nombre-nota]]`
Tags	`#proyecto`, `#idea`, `#research`
Nombres de archivo	`proyecto-nuevo.md` (minúsculas, guiones)
Fechas	`2026-04-28` (formato ISO)

Sección 5: Estructura de carpetas
Le explica a Claude exactamente dónde guardar cada tipo de cosa.

Sección 6: Reglas de comportamiento
- Crear siempre enlaces entre notas relacionadas - Capturar ideas automáticamente sin esperar confirmación - Incluir el campo "□ Una sola cosa" en cada nota diaria - Mantener el sistema de memoria actualizado

¿Cómo se usa?

Vos editás `CLAUDE.md` con tu información real (nombre, proyectos, etc.) una sola vez. A partir de ahí, Claude lo lee automáticamente al inicio de cada sesión.

5.2 .claude/settings.json — Los permisos del agente

Este archivo controla qué operaciones puede hacer Claude con tus archivos. Es como una lista de accesos.

```
{
  "permissions": {
    "allow": [
      "Write",
      "Read",
      "Edit",
      "Glob",
      "Grep"
    ],
    "deny": []
  }
}
```

¿Qué significa cada permiso?

Permiso	¿Qué puede hacer?
`Read`	Leer archivos existentes
`Write`	Crear archivos nuevos
`Edit`	Modificar archivos existentes
`Glob`	Buscar archivos por patrón (ej: todos los `.md`)
`Grep`	Buscar texto dentro de archivos

¿Por qué es importante?
Por seguridad. Si Claude solo tiene permiso para leer y escribir archivos Markdown en tu vault, no puede ejecutar programas, modificar configuraciones del sistema ni hacer nada fuera de su área. El control granular de permisos es una buena práctica cuando se trabaja con agentes de IA.

5.3 La carpeta memory/ — La memoria del agente

Esta es la innovación más importante del proyecto. En lugar de que cada sesión de IA empiece desde cero, el agente guarda información en archivos Markdown y la recupera en

sesiones futuras.

Tipos de memorias

user_*.md — Memorias sobre vos
El agente guarda cosas que aprendió sobre vos: - Tu área de trabajo - Tus preferencias - Tu nivel de experiencia en distintos temas - Cómo explicarte mejor las cosas

project_*.md — Memorias sobre proyectos
Por cada proyecto activo, el agente puede guardar: - Estado actual - Decisiones tomadas - Próximos pasos - Contexto importante

feedback_*.md — Memorias sobre preferencias de trabajo
Si le decís "no me gusta cuando hacés X" o "prefiero que respondas así", el agente lo guarda para no repetir el mismo error.

El índice MEMORY.md

Es un archivo que lista todas las memorias existentes. El agente lo lee primero para saber qué recuerda, y luego carga solo lo relevante para la conversación actual.

```
## Índice de Memorias

- [user_perfil.md](memory/user_perfil.md) — Perfil y contexto del usuario
- [project_tesis.md](memory/project_tesis.md) — Estado de tesis en curso
- [feedback_estilo.md](memory/feedback_estilo.md) — Preferencias de comuni
```

PARTE 6 — Las plantillas explicadas

6.1 Plantilla de Nota Diaria (daily-note.md)

Se crea una nueva nota por día. Contiene:

```
---
fecha: 2026-04-28
tags: [daily]
---

# □ 2026-04-28

## □ Una sola cosa
> La tarea más importante del día (solo una)

## □ Objetivos del día
- [ ] Objetivo 1
- [ ] Objetivo 2
- [ ] Objetivo 3

## □ Braindump
(Escribí todo lo que tenés en la cabeza, sin filtro)

## □ Completado
- (Se completa al final del día)

## □ Ideas que surgieron

## □ Referencias
```

El campo "□ Una sola cosa" es obligatorio. La idea es que cada día tengas una prioridad máxima clara, no diez cosas importantes.

6.2 Plantilla de Proyecto (proyecto.md)

```
---
estado: activo
inicio: 2026-04-01
objetivo: 2026-06-30
tags: [proyecto, #estado/activo]
---

# Nombre del Proyecto

## 📋 Descripción
¿De qué se trata? ¿Por qué importa?

## 🎯 Objetivos
- Objetivo específico 1
- Objetivo específico 2

## 📅 Decisiones clave
| Fecha | Decisión | Justificación |
|-------|----------|---------------|

## ⏭ Próximos pasos
- [ ] Tarea pendiente 1
- [ ] Tarea pendiente 2

## 📊 Estado actual
Resumen del progreso...

## 🔗 Relacionado
- [[personas/nombre]]
- [[research/tema]]
```

6.3 Plantilla de Persona (persona.md)

Para guardar información sobre personas relevantes (colegas, mentores, contactos):

```
---
tags: [persona]
---

# Nombre Apellido

## 📍 Contexto
¿Quién es? ¿Cómo lo conocés?

## 💼 Rol
¿En qué trabaja? ¿Cómo se relaciona con tus proyectos?

## 📝 Notas
Observaciones, preferencias, cosas a recordar

## 🔗 Relacionado
- [[proyectos/proyecto-comun]]
```

6.4 Plantilla de Research (research.md)

Para temas de investigación o aprendizaje:

```
---
tags: [research, #estado/activo]
---

# Tema de Investigación

## 🎯 Pregunta principal

## 📚 Fuentes
- Fuente 1
- Fuente 2

## 🧠 Conceptos clave

## 💡 Conclusiones e insights

## 🔗 Relacionado
```

PARTE 7 — Cómo instalar y configurar el sistema

Requisitos previos

Antes de empezar necesitás instalar:

1. **Obsidian** — descargarlo en obsidian.md (gratuito)
2. **VS Code** — descargarlo en code.visualstudio.com (gratuito)
3. **Claude Code** — extensión de VS Code, buscala en el Marketplace
4. **Cuenta en Anthropic** — anthropic.com (necesaria para usar Claude)
5. **Plugin Templater para Obsidian** — desde el marketplace de plugins de Obsidian

Instalación paso a paso

Paso 1: Clonar el repositorio

Abrí una terminal y ejecutá:

```
git clone https://github.com/KevinLarguia/obsidian-claude-agent-IA-SecondB
```

O si no usás git, descargá el ZIP desde GitHub y descomprimilo.

Paso 2: Abrir como vault en Obsidian

1. Abrí Obsidian
2. Seleccioná “Open folder as vault”
3. Navegá hasta la carpeta que descargaste
4. Hacé click en “Open”

Ahora Obsidian reconoce esa carpeta como tu vault.

Paso 3: Configurar Templater

1. En Obsidian, andá a Configuración → Plugins de la comunidad
2. Buscá “Templater” e instalalo
3. En la configuración de Templater, apuntá la carpeta de templates a `templates/`

Paso 4: Editar CLAUDE.md con tu información

Abrí `CLAUDE.md` en cualquier editor de texto y completá las secciones con tu información real:

```
## Perfil del usuario
- **Nombre:** Tu nombre aquí
- **Educación:** Tu carrera/formación
- **Objetivos:** Lo que estás buscando lograr
- **Proyectos activos:**
  - Proyecto 1: descripción breve
  - Proyecto 2: descripción breve
```

Este paso es crucial. Cuanto más detallado sea tu perfil, mejor entiende el agente tu contexto.

Paso 5: Abrir el vault en VS Code

1. Abrí VS Code
2. Andá a File → Open Folder
3. Seleccioná la misma carpeta del vault

Paso 6: Activar Claude Code

1. En VS Code, abrí la paleta de comandos (Ctrl+Shift+P)
2. Buscá “Claude Code” y abrilo
3. Iniciá sesión con tu cuenta de Anthropic

Paso 7: Primera sesión

Con Claude Code abierto, escribile:

```
Lee el archivo CLAUDE.md y luego créame la nota diaria de hoy.
```

Si todo funciona correctamente, Claude va a:

1. Leer `CLAUDE.md` y entender tu perfil
2. Crear una nueva nota en `daily-notes/` con la fecha de hoy
3. Completar la estructura de la plantilla

PARTE 8 — Cómo usarlo en el día a día

Flujo de trabajo típico

Al inicio del día

```
Abrís VS Code → Claude Code lee CLAUDE.md automáticamente

Le escribís: "Buenos días, creame la nota de hoy"

Claude crea daily-notes/2026-04-28.md con toda la estructura
```

Durante el día — Captura de ideas

Si en medio de una reunión se te ocurre algo:

```
"Idea: podría automatizar el proceso X usando Y"
```

Claude automáticamente guarda eso en `inbox/` o en `ideas/` sin que vos tengas que especificarlo. La captura es inmediata.

Para gestionar proyectos

```
"¿Cuál es el estado de mi proyecto de tesis?"
```

Claude busca en `proyectos/tesis.md` y te da un resumen con los próximos pasos.

```
"Actualizá el estado de la tesis: terminé el capítulo 2"
```

Claude edita el archivo directamente.

Para conectar conocimiento

```
"¿Hay algo en mi vault relacionado con machine learning?"
```

Claude busca en todos tus archivos y te muestra las notas relevantes con sus conexiones.

Al final del día

```
"Hacé el cierre del día: completé los objetivos 1 y 2,
el 3 quedó pendiente para mañana"
```

Claude actualiza tu nota diaria, mueve la tarea pendiente y puede sugerir qué priorizar mañana.

PARTE 9 — Conceptos técnicos para entender mejor

¿Qué es un LLM?

LLM significa “Large Language Model” (Modelo de Lenguaje Grande). Es el tipo de IA que está detrás de Claude, ChatGPT, etc.

Estos modelos son entrenados con enormes cantidades de texto y aprenden a predecir cuál es la continuación más probable de un texto dado. En la práctica, esto resulta en sistemas que parecen "entender" el lenguaje y pueden generar texto coherente, responder preguntas, razonar sobre problemas, etc.

¿Qué es un agente de IA?

Un agente de IA es un LLM que, además de generar texto, puede ejecutar acciones en el mundo real. En este proyecto, el agente puede:

- **Leer** archivos de tu vault
- **Escribir** nuevos archivos
- **Editar** archivos existentes
- **Buscar** contenido dentro de archivos

La diferencia con un chatbot normal es que el agente tiene herramientas (tools) que le permiten interactuar con sistemas externos, no solo generar texto.

¿Qué es el context window?

El context window (ventana de contexto) es la cantidad de texto que un LLM puede procesar de una vez. Es como la memoria de trabajo del modelo: solo "recuerda" lo que entra en esa ventana.

El sistema de memoria en `memory/` es una solución a este problema: en lugar de cargar todo el vault en el contexto (lo cual sería imposible para vaults grandes), el agente carga selectivamente solo la información relevante para la conversación actual.

¿Qué es YAML frontmatter?

Es metadata estructurada al inicio de los archivos Markdown. Se delimita con tres guiones:

```yaml
---
fecha: 2026-04-28
estado: activo
tags: [proyecto, trabajo]
---
```

Obsidian y otras herramientas pueden leer esta metadata para filtrar, ordenar y buscar notas. Por ejemplo, podés ver todas las notas con `estado: activo` de un vistazo.

¿Qué son los wikilinks?

Los wikilinks son la forma de linkear notas en Obsidian usando doble corchete: `[[nombre-de-nota]]`. Cuando hacés click en un wikilink, Obsidian te lleva a esa nota. Así se construye la red de conocimiento.

Claude también crea estos links automáticamente cuando escribe notas, conectando ideas relacionadas.

PARTE 10 — Beneficios y casos de uso reales

¿Para qué tipo de persona es esto útil?

Este sistema es especialmente valioso para:

Estudiantes y académicos: Gestionar bibliografía, conectar conceptos de distintas materias, preparar síntesis para exámenes, hacer seguimiento de tesis o trabajos de investigación.

Profesionales del conocimiento: Data scientists, desarrolladores, consultores que necesitan registrar aprendizajes técnicos, decisiones de diseño, contexto de proyectos.

Emprendedores: Gestionar múltiples proyectos simultáneamente, capturar ideas, hacer seguimiento de contactos clave.

Cualquier persona: Que quiera tener un sistema de notas que no solo archive información sino que la conecte y la haga útil.

Casos de uso concretos

Caso 1: Estudiante de Data Science

```
Usuario: "Estoy aprendiendo transformers. ¿Qué tengo en mi vault sobre
esto
y qué debería agregar?"

Claude: Lee research/transformers.md, ve que hay gaps en la sección de
attention mechanisms, sugiere recursos específicos y crea una nota de
seguimiento en research/.
```

Caso 2: Reunión importante

```
Usuario: "Mañana tengo reunión con el cliente X sobre el proyecto Y.
Dame un brief de lo que necesito saber."

Claude: Lee personas/cliente-x.md y proyectos/proyecto-y.md, genera
un resumen ejecutivo con el contexto relevante, decisiones pasadas
y próximos pasos acordados.
```

Caso 3: Captura de ideas

```
Usuario: "Idea: hacer un sistema de review automático para PRs usando
Claude API"

Claude: Crea automáticamente inbox/2026-04-28-review-prs.md con la
idea capturada, tags relevantes y links a notas relacionadas.
```

PARTE 11 — Buenas prácticas y recomendaciones

Para sacarle el máximo provecho

1. Completá bien CLAUDE.md desde el primer día
Cuanto más contexto tenga el agente sobre vos, mejores serán sus respuestas. No seas vago: detallá tus proyectos, tus objetivos, cómo te gusta trabajar.

2. Sé consistente con las convenciones
Usá siempre los mismos formatos de nombre, los mismos tags, la misma estructura. El sistema funciona mejor cuando hay consistencia.

3. Capturá sin filtrar
No te preocupes si algo parece una "mala idea" o está incompleto. Mandalo a `inbox/` y revisalo después. La captura rápida es clave.

4. Revisá las memorias periódicamente
Cada tanto, revisá los archivos en `memory/` para asegurarte de que el agente tiene información actualizada sobre vos.

5. Usá las notas diarias religiosamente
El hábito de la nota diaria es el pegamento del sistema. Te da un punto de inicio y cierre de día, y acumula un registro valioso.

Lo que no hacer

No pongas secretos en las memorias: Si tu vault está en una computadora compartida, recordá que los archivos de memoria son texto plano y cualquiera puede leerlos.

No confíes ciegamente: El agente puede cometer errores. Siempre revisá lo que escribe antes de asumirlo como correcto.

No sobrecargues CLAUDE.md: Si se vuelve demasiado largo, el agente va a tener problemas para procesar todo. Priorizá la información más relevante.

PARTE 12 — Glosario rápido

Término	Significado
LLM	Large Language Model — modelo de IA que procesa lenguaje natural
Agente	LLM con capacidad de ejecutar acciones (leer/escribir archivos)
Vault	Carpeta raíz de Obsidian que contiene todas tus notas
Context Engineering	Arte de estructurar el contexto que se le da a una IA
Context Window	Cantidad máxima de texto que un LLM puede procesar de una vez
Markdown	Lenguaje de marcado simple para formatear texto
YAML Frontmatter	Metadata estructurada al inicio de archivos Markdown
Wikilink	Enlace entre notas usando `[[doble corchete]]`
System Prompt	Instrucciones iniciales que configuran el comportamiento de una IA
Memoria persistente	Información guardada entre sesiones para no perder contexto
Plugin Templater	Plugin de Obsidian para crear notas desde plantillas automáticamente
allowlist	Lista de permisos permitidos (lo opuesto a blocklist/denylist)
Inbox	Bandeja de entrada — lugar de captura rápida sin organizar
Tag	Etiqueta para clasificar y buscar notas

PARTE 13 — Preguntas frecuentes

¿Necesito saber programar para usar esto?
No. El sistema está diseñado para ser usado en lenguaje natural. Lo único "técnico" que necesitás entender es cómo funciona el Markdown básico, que aprendés en 10 minutos.

¿Mis notas son privadas?
Tus archivos viven en tu computadora. Cuando usás Claude Code, el contenido de tus archivos se envía a los servidores de Anthropic para ser procesado. Revisá la política de privacidad de Anthropic si eso es relevante para vos.

¿Puedo usar esto con otros modelos de IA (no solo Claude)?
El sistema está diseñado específicamente para Claude Code. Las convenciones y CLAUDE.md están optimizadas para Claude. Adaptarlo a otras herramientas requeriría modificaciones.

¿Qué pasa si tengo un vault enorme con miles de notas?
El sistema de memoria en `memory/` está diseñado precisamente para esto. Claude no carga todo el vault en cada conversación, sino que accede selectivamente a lo que necesita.

¿Puedo usarlo en equipo?
El proyecto está pensado para uso individual. Usar un vault compartido en equipo requeriría adaptaciones importantes en las convenciones y el sistema de memoria.

¿Tengo que pagar para usar Claude Code?
Claude Code requiere una cuenta de Anthropic. Anthropic tiene planes de pago según el uso. Verificá los precios actuales en anthropic.com.

¿Qué tan diferente es esto de simplemente usar Claude en el navegador?
La diferencia fundamental es que Claude en el navegador no tiene acceso a tus archivos locales y cada sesión empieza desde cero. Este sistema le da a Claude acceso persistente a toda tu base de conocimiento y memoria entre sesiones.

CONCLUSIÓN

Este proyecto combina tres ideas poderosas:

1. **Gestión del conocimiento personal** (Obsidian como segundo cerebro)
2. **IA con acceso a contexto real** (Claude Code leyendo tu vault)
3. **Memoria persistente entre sesiones** (archivos en `memory/`)

El resultado es un sistema donde la IA no es un asistente genérico, sino un colaborador que conoce tu contexto, tus proyectos y tus preferencias. Cada sesión es más útil que la anterior porque el agente sigue acumulando conocimiento sobre cómo trabajás.

Para la comunidad de Data Science e IA, este proyecto también sirve como ejemplo práctico de:

- Prompt engineering / context engineering avanzado
- Arquitectura de agentes con memoria
- Diseño de sistemas de gestión del contexto
- Control de acceso granular para agentes de IA

Es, en esencia, un prototipo funcional de cómo se construyen sistemas de IA útiles en producción.

Documentación generada el 28 de abril de 2026
Repositorio original: github.com/KevinLarguia/obsidian-claude-agent-IA-SecondBrain-